

```
In [1]: import cv2
import numpy as np
import matplotlib.pyplot as plt

print("Libraries Imported Successfully")
```

Libraries Imported Successfully

```
In [2]: cap = cv2.VideoCapture(0)

if cap.isOpened():
    print("Webcam Connected")
else:
    print("Webcam Not Found")

cap.release()
```

Webcam Connected

```
In [3]: cap = cv2.VideoCapture(0)

ret, frame = cap.read()

gray = cv2.cvtColor(
    frame,
    cv2.COLOR_BGR2GRAY
)

corners = cv2.goodFeaturesToTrack(
    gray,
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7
)

corners = np.int32(corners)

for corner in corners:

    x, y = corner.ravel()

    cv2.circle(
        frame,
        (x, y),
        4,
        (0,255,0),
        -1
    )

cv2.imshow(
    "Shi-Tomasi Corners",
    frame
)

cv2.waitKey(0)

cv2.destroyAllWindows()

cap.release()
```

```
In [4]: cap = cv2.VideoCapture(0)

ret, old_frame = cap.read()

old_gray = cv2.cvtColor(
    old_frame,
    cv2.COLOR_BGR2GRAY
)

feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

lk_params = dict(
    winSize=(15,15),
```

```

    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p0 = cv2.goodFeaturesToTrack(
    old_gray,
    mask=None,
    **feature_params
)

mask = np.zeros_like(old_frame)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    frame_gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    p1, st, err = cv2.calcOpticalFlowPyrLK(
        old_gray,
        frame_gray,
        p0,
        None,
        **lk_params
    )

    if p1 is not None:

        good_new = p1[st==1]
        good_old = p0[st==1]

        for new, old in zip(
            good_new,
            good_old
        ):

            a,b = new.ravel()
            c,d = old.ravel()

            mask = cv2.line(
                mask,
                (int(a),int(b)),
                (int(c),int(d)),
                (0,255,0),
                2
            )

            frame = cv2.circle(
                frame,

```

```

        (int(a),int(b)),
        4,
        (0,0,255),
        -1
    )

img = cv2.add(
    frame,
    mask
)

cv2.imshow(
    "Object Tracking",
    img
)

old_gray = frame_gray.copy()

p0 = good_new.reshape(
    -1,
    1,
    2
)

if cv2.waitKey(30) & 0xFF == 27:
    break

cap.release()

cv2.destroyAllWindows()

```

In [5]:

```

import numpy as np

for new, old in zip(
    good_new,
    good_old
):

    a,b = new.ravel()

    c,d = old.ravel()

    dx = a - c
    dy = b - d

    magnitude = np.sqrt(
        dx**2 + dy**2
    )

    direction = np.degrees(
        np.arctan2(
            dy,
            dx
        )
    )

    print(
        f"Magnitude={magnitude:.2f}, Direction={direction:.2f}"
    )

```

Magnitude=0.32, Direction=-102.98
 Magnitude=0.12, Direction=-90.77
 Magnitude=0.44, Direction=-111.66
 Magnitude=25.14, Direction=-70.33
 Magnitude=5.89, Direction=-5.88

```

In [6]: displacements = []

for new, old in zip(
    good_new,
    good_old
):

    a,b = new.ravel()

    c,d = old.ravel()

    distance = np.sqrt(
        (a-c)**2 +
        (b-d)**2
    )

    displacements.append(
        distance
    )

print(
    "Average Displacement:",
    np.mean(displacements)
)

```

Average Displacement: 6.381387

```

In [7]: cap = cv2.VideoCapture(0)

ret, frame1 = cap.read()

prvs = cv2.cvtColor(
    frame1,
    cv2.COLOR_BGR2GRAY
)

hsv = np.zeros_like(frame1)

hsv[...,1] = 255

while True:

    ret, frame2 = cap.read()

    if not ret:
        break

    nxt = cv2.cvtColor(
        frame2,
        cv2.COLOR_BGR2GRAY
    )

    flow = cv2.calcOpticalFlowFarneback(
        prvs,

```

```

        nxt,
        None,
        0.5,
        3,
        15,
        3,
        5,
        1.2,
        0
    )

    mag, ang = cv2.cartToPolar(
        flow[...,0],
        flow[...,1]
    )

    hsv[...,0] = ang*180/np.pi/2

    hsv[...,2] = cv2.normalize(
        mag,
        None,
        0,
        255,
        cv2.NORM_MINMAX
    )

    rgb = cv2.cvtColor(
        hsv,
        cv2.COLOR_HSV2BGR
    )

    cv2.imshow(
        "Farneback Tracking",
        rgb
    )

    prvs = nxt

    if cv2.waitKey(30) & 0xFF == 27:
        break

cap.release()

cv2.destroyAllWindows()

```

```

In [8]: print("Lucas-Kanade")
print("- Sparse Optical Flow")
print("- Tracks selected features")
print("- Faster")
print("- Lower computational cost")

print("\nFarneback")
print("- Dense Optical Flow")
print("- Tracks all pixels")
print("- Rich motion information")
print("- Higher computational cost")

```

Lucas-Kanade

- Sparse Optical Flow
- Tracks selected features
- Faster
- Lower computational cost

Farneback

- Dense Optical Flow
- Tracks all pixels
- Rich motion information
- Higher computational cost

```
In [9]: print("""  
OBSERVATIONS  
  
1. Shi-Tomasi successfully detected feature points.  
2. Lucas-Kanade tracked object trajectories effectively.  
3. Motion vectors represented object movement.  
4. Farneback generated dense motion fields.  
5. Tracking accuracy decreased during fast motion.  
6. Illumination changes affected optical flow estimation.  
7. Camera movement introduced additional motion vectors.  
""")
```

OBSERVATIONS

1. Shi-Tomasi successfully detected feature points.
2. Lucas-Kanade tracked object trajectories effectively.
3. Motion vectors represented object movement.
4. Farneback generated dense motion fields.
5. Tracking accuracy decreased during fast motion.
6. Illumination changes affected optical flow estimation.
7. Camera movement introduced additional motion vectors.